SendToEmail

Security Review

March 5, 2026

Prepared by: Valves Security

Lead Auditors: Vesko210 | Merulez99

Contents

1 About Valves Security 3
2 Disclaimer 3
3 Risk Classification 3
3.0.1 Impact . . . 3
3.0.2 Likelihood . . . 3
4 Protocol Summary 4
5 Executive Summary 4
6 Findings 5
6.1 Medium . . . 5
6.1.1 Fee Bypass via Wallet-Count Cycling . . . 5
6.1.2 Missing `AlreadyPremium` Guard . . . 6
6.1.3 Wallet Binding Griefing . . . 6
6.1.4 Identity Squatting via `register` Front-Running . . . 7
6.2 Low/Informational . . . 8
6.2.1 Fee-on-Transfer Tokens Underpay Treasury . . . 8
6.2.2 `_send(treasury, 0)` Revert Risk in Zero-Fee Campaigns . . . 9
6.2.3 Zero-Fee `safeTransferFrom(0)` Reverts on Strict Tokens . . . 10
6.2.4 CEI violation - wallet count incremented after external call . . . 11

1 About Valves Security

Valves Security is a Web3 security team founded by Veselin and Valeri. With deep experience across EVM-based protocols, Cosmos (Go) chains, and native Rust systems, we help projects identify and eliminate critical vulnerabilities. With a strong background and a strong team of freelancers we can asure a top quality audit.

Book a Security Review at valvessecurity.com or message us on X Valves Security

2 Disclaimer

The Valves team makes all effort to find as many vulnerabilities in the code in the given time period. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation. We stand by our process, yet encourage follow-up audits and meaningful bug bounty incentives for continued whitehat protection.

3 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	**Critical**	**High**	**Medium**
Likelihood: Medium	**High**	**Medium**	**Low**
Likelihood: Low	**Medium**	**Low**	**Low**

3.0.1 Impact

- High - leads to a significant material loss of assets in the protocol or a group of users
- Medium - leads to a moderate material loss of assets in the protocol, moderately harms a group of users or to some disruption of the protocol's functionality
- Low - No funds at risk

3.0.2 Likelihood

- High - attack path is possible with reasonable assumptions. The issue is almost certain to happen.
- Medium - only a conditionally incentivized attack vector, but still relatively likely
- Low - has unlikely assumptions or requires a significant stake by the attacker

4 Protocol Summary

SendToEmail is an on-chain identity registry that maps hashed email addresses to multi-chain wallet addresses, allowing anyone to look up a recipient's wallet by their email hash without exposing the email itself. Users register an identity, then bind wallet addresses for up to 10 supported networks, with the first two slots free and subsequent slots requiring a fee paid in ETH or whitelisted ERC-20 tokens. A premium tier removes per-slot fees entirely, and all admin capabilities are limited to fee and treasury management, the protocol enforces that only the identity owner can modify their own record.

5 Executive Summary

Overview

Project	SendToEmail
Commit Hash	09ee1c2846ae82959e0efcca41243302f616589f
Timeline	01/03/2026 – 04/03/2026

Scope

Id	Files in scope
1	src/*

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	4
Low/Info Risk	4

6 Findings

6.1 Medium

6.1.1 Fee Bypass via Wallet-Count Cycling

Description Fee eligibility checks current `_walletCounts[emailHash] < FREE_SLOTS`. `removeWallet` decrements `_walletCounts`, so users can repeatedly drop below threshold and regain free-tier eligibility.

Impact

- Users can avoid expected wallet slot fees.
- Loss of revenue for the protocol.

Proof of Concept

```
function test_feeBypassCycling_poc() public {
    _registerAlice();

    uint256 aliceBefore = alice.balance;
    uint256 treasuryBefore = treasury.balance;

    // use 2 free slots
    vm.prank(alice);
    s.setWallet(aliceHash, ISendToEmail.Network.EVM, _evmWallet(alice));
    vm.prank(alice);
    s.setWallet(aliceHash, ISendToEmail.Network.Solana, hex"aabb");
    assertEq(s.walletCount(aliceHash), 2);

    // cycle networks 2..9, always staying below FREE_SLOTS right before each paid slot
    for (uint8 i = 2; i < 10; i++) {
        vm.prank(alice);
        s.removeWallet(aliceHash, ISendToEmail.Network(i - 2));

        bytes memory w = abi.encodePacked(bytes1(i), bytes20(alice));
        vm.prank(alice);
        s.setWallet(aliceHash, ISendToEmail.Network(i), w);
    }

    // no ETH fee paid despite repeated wallet additions over time
    assertEq(alice.balance, aliceBefore);
    assertEq(treasury.balance, treasuryBefore);
}
```

Recommended Mitigation

- Add monotonic lifetime counter (`_walletCountsTotal`).
- Use `_walletCountsTotal` for fee gating.
- Keep `_walletCounts` only for active-wallet UX/accounting.

6.1.2 Missing `AlreadyPremium` Guard

Description `activatePremium` / `activatePremiumWithToken` do not check if premium is already active before charging.

Impact

- Duplicate calls charge repeatedly for same state.
- Can happen from retries or broken UI paths.

Proof of Concept

```
function test_activatePremium_doublePayment_poc() public {
    _registerAlice();

    vm.prank(alice);
    s.activatePremium{value: PREMIUM_FEE}(aliceHash);
    assertTrue(s.isPremium(aliceHash));

    uint256 treasuryBefore = treasury.balance;

    vm.prank(alice);
    s.activatePremium{value: PREMIUM_FEE}(aliceHash);

    assertEq(treasury.balance, treasuryBefore + PREMIUM_FEE);
    assertTrue(s.isPremium(aliceHash));
}
```

Recommended Mitigation

Add:

```
if (_premium[emailHash]) revert AlreadyPremium();
```

to both premium activation paths.

6.1.3 Wallet Binding Griefing

Description `setWallet` / `setWalletWithToken` enforce uniqueness but do not verify wallet ownership by caller. Attackers can bind victim addresses first.

Impact

- Victims can be blocked from binding their own address.

Proof of Concept

```
function test_walletGriefing_poc() public {
    bytes memory victimsEVMAddress = _evmWallet(alice);

    // attacker registers identity
    _registerBob();

    // attacker binds victim's address
    vm.prank(bob);
    s.setWallet(bobHash, ISendToEmail.Network.EVM, victimsEVMAddress);

    // victim registers later
    _registerAlice();

    // victim is blocked
    vm.expectRevert(ISendToEmail.WalletAlreadyBound.selector);
    vm.prank(alice);
    s.setWallet(aliceHash, ISendToEmail.Network.EVM, victimsEVMAddress);
}
```

Recommended Mitigation

- Require ownership proofs before binding:
  - EVM: ECDSA signature from wallet.
  - Non-EVM: native proof or trusted attestation.
- Update reverse mapping only after successful verification.

6.1.4 Identity Squatting via `register` Front-Running

Description `register(bytes32 emailHash)` is permissionless and only checks whether `emailHash` is already registered.
There is no caller-to-email ownership proof and no commit-reveal, so mempool observers can front-run the same hash.

Impact

- Permanent denial of registration for legitimate owner.
- Attacker can bind wallets under hijacked identity.

- Attack cost is low (gas only).

Proof of Concept

```
function test_identitySquatting_poc() public {
    bytes32 target = s.hashEmail("victim@example.com");

    // attacker front-runs victim and registers first
    vm.prank(bob);
    s.register(target);

    // victim cannot register their own email
    vm.expectRevert(ISendToEmail.AlreadyRegistered.selector);
    vm.prank(alice);
    s.register(target);

    // attacker owns the victim's identity
    assertEq(s.ownerOf(target), bob);
}
```

Recommended Mitigation

- Add commit-reveal registration where the registration is first checked off-chain
- Optionally add recovery/dispute flow for obvious squatting cases (only Admin functionality).

6.2 Low/Informational

6.2.1 Fee-on-Transfer Tokens Underpay Treasury

Description Protocol assumes requested token fee equals received token amount. With fee-on-transfer tokens, treasury receives less than configured.

Impact

- Silent revenue/accounting mismatch that can lead to moderate loss after enough time has passed.

Proof of Concept

```
function test_feeOnTransferToken_underpaysTreasury_poc() public {
    FeeOnTransferToken fot = new FeeOnTransferToken();
    uint256 configuredWalletFee = 1000;

    vm.prank(owner);
    s.setTokenFees(address(fot), configuredWalletFee,
        configuredWalletFee);

```

```
    _registerAlice();
    fot.mint(alice, 10_000);

    vm.prank(alice);
    s.setWalletWithToken(aliceHash, ISendToEmail.Network.EVM, _evmWallet(alice), address(fot));
    vm.prank(alice);
    s.setWalletWithToken(aliceHash, ISendToEmail.Network.Solana, hex"aabb", address(fot));

    vm.prank(alice);
    fot.approve(address(s), type(uint256).max);

    vm.prank(alice);
    s.setWalletWithToken(aliceHash, ISendToEmail.Network.Bitcoin, hex"ccdd", address(fot));

    // configured fee is 1000, treasury receives 990
    assertEq(fot.balanceOf(treasury), 990);
    assertLt(fot.balanceOf(treasury), configuredWalletFee);
}
```

Recommended Mitigation

- Either disallow fee-on-transfer tokens, or check balance deltas (`before`/`after`) and revert on shortfall.

6.2.2 `_send(treasury, 0)` Revert Risk in Zero-Fee Campaigns

Description `_send` performs low-level call even when amount is zero.
If treasury rejects zero-value calls, fee=0 paths can revert unexpectedly.

Impact

- Zero-fee campaigns can break operations.

Proof of Concept

```
function test_zeroFees_rejectingTreasury_reverts_setWallet_poc() public {
    RejectETH rejectingTreasury = new RejectETH();
    SendToEmail local = new SendToEmail(address(rejectingTreasury), 0, 0, owner);

    bytes32 h = local.hashEmail("user@example.com");
    address user = makeAddr("user");
    vm.deal(user, 1 ether);

    vm.prank(user);
```

```
    local.register(h);

    vm.prank(user);
    local.setWallet(h, ISendToEmail.Network.EVM, _evmWallet(user));
    vm.prank(user);
    local.setWallet(h, ISendToEmail.Network.Solana, hex"aabb");

    vm.expectRevert(ISendToEmail.TransferFailed.selector);
    vm.prank(user);
    local.setWallet(h, ISendToEmail.Network.Bitcoin, hex"ccdd");
}
```

Recommended Mitigation Short-circuit zero sends in `_send` the same that is done with the excess value returned to the users:

```
if (walletFee > 0) {
    _send(treasury, walletFee);
}
```

```
if (premiumFee > 0) {
    _send(treasury, premiumFee);
}
```

6.2.3 Zero-Fee `safeTransferFrom(0)` Reverts on Strict Tokens

Description Token fee paths always call `safeTransferFrom(..., fee)` even when `fee == 0`. Some tokens revert on zero-amount transfers.

Impact

- Token-specific DoS for zero-fee configs.
- Affects wallet/premium token flows for strict tokens.

Proof of Concept

```
function
    test_zeroFeeToken_strictTransfer_reverts_setWalletWithToken_poc()
    public {
    StrictToken strict = new StrictToken();

    vm.prank(owner);
    s.setTokenFees(address(strict), 0, USDC_PREMIUM_FEE);

    _registerAlice();
    strict.mint(alice, 1000 ether);

    vm.prank(alice);
```

```
    s.setWalletWithToken(aliceHash, ISendToEmail.Network.EVM,
        _evmWallet(alice), address(strict));
    vm.prank(alice);
    s.setWalletWithToken(aliceHash, ISendToEmail.Network.Solana, hex"
        aabb", address(strict));

    vm.startPrank(alice);
    strict.approve(address(s), 1 ether);
    vm.expectRevert();
    s.setWalletWithToken(aliceHash, ISendToEmail.Network.Bitcoin, hex"
        ccdd", address(strict));
    vm.stopPrank();
}
```

Recommended Mitigation Guard token transfers with:

```
if (fee > 0) { ... }
```

6.2.4 CEI violation - wallet count incremented after external call

Description

In both `setWallet` and `setWalletWithToken`, `_walletCounts[emailHash]++` is executed **after** the external interaction (`_collectFee` / `_collectTokenFee`), violating the Checks-Effects-Interactions pattern. The comment in the code incorrectly identifies this block as following CEI.

No direct risk but good to be fixed.

Recommendation

Move `_walletCounts[emailHash]++` to before the fee collection call:

```
if (!isUpdate) {
    _walletCounts[emailHash]++;   // effect first
    _collectFee(emailHash);       // then interaction
}
```